

우리금융 실습 5주차

SVM, 군집분석, 차원축소

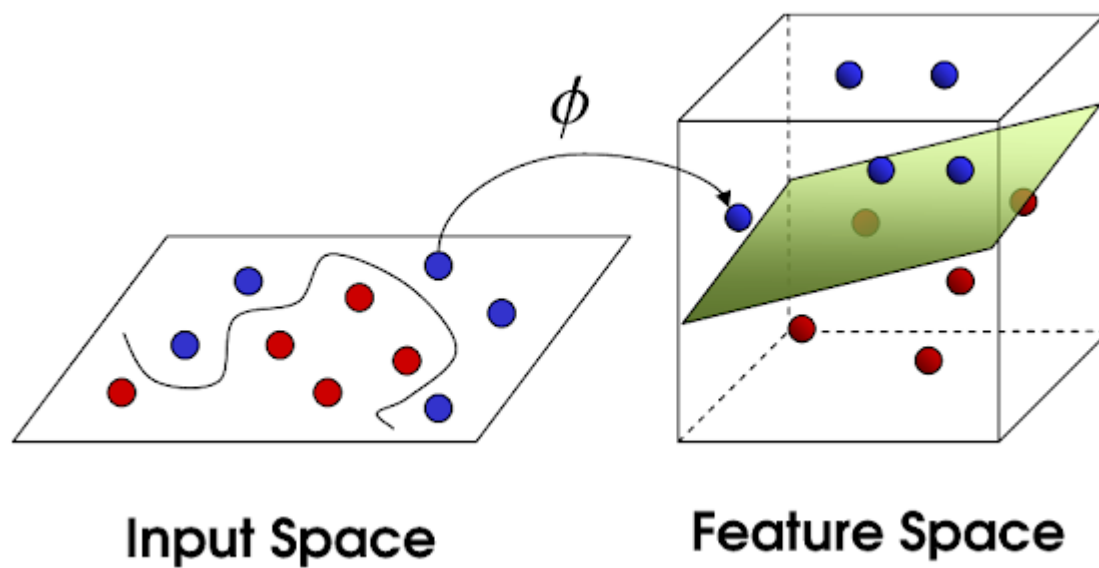
김초은, 박세현

목차

- Support Vector Machine
- 군집분석
 - 계층적 군집분석
 - K-평균 군집분석
- 차원축소
 - PCA
 - MDS
 - T-SNE

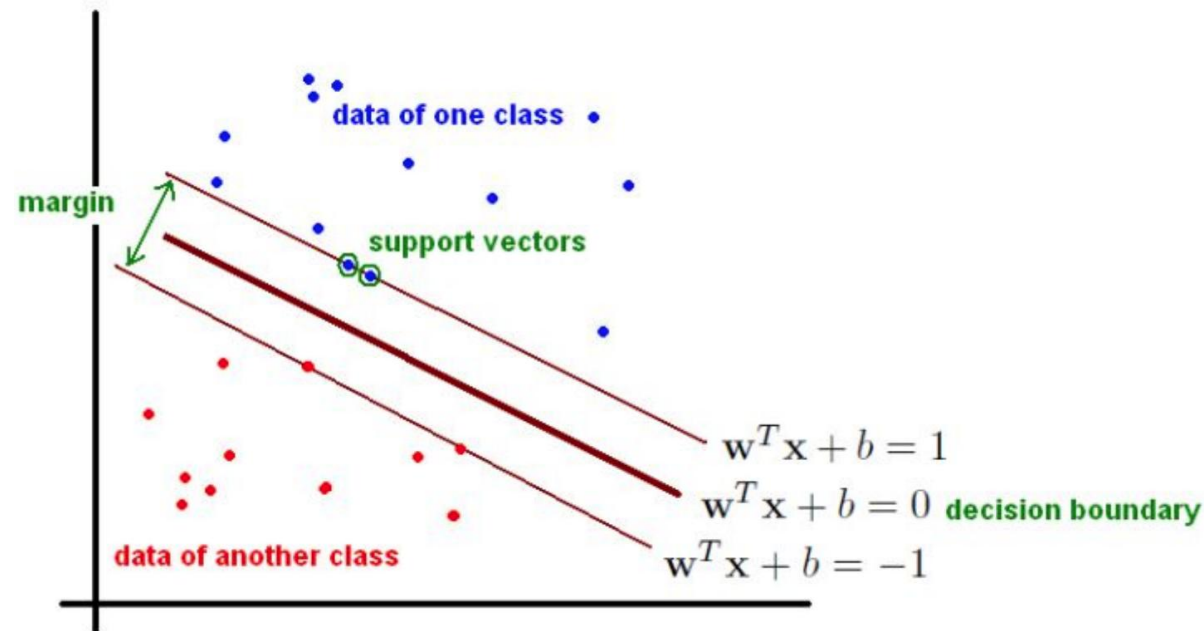
Support Vector Machine

- 선형 분리가 불가능한 복잡한 데이터 분류하기
- 커널 함수를 사용해서 고차원 공간으로 보낸 후 선형 분리하는 방법.



Support Vector Machine

- Margin: 결정 경계와 가장 가까운 점까지의 거리
- Margin의 합을 최대화하는 선형 분류기를 찾음.
- Soft margin: 일부 샘플은 경계를 넘어도 허용해, 분류 오류를 감수하는 기법.



Support Vector Machine

- Data load and split



```
# Load German credit data
german = fetch_openml("credit-g", version=1, as_frame=True)
german_df = german['data']
german_y= german.target

# Encode categorical data (convert strings to numbers)
label_encoder = LabelEncoder()
X_encoded = german_df.apply(label_encoder.fit_transform)
y_encoded = label_encoder.fit_transform(german_y)

# Split the data (train/test sets)
X_train, X_test, y_train, y_test = train_test_split(
    X_encoded, y_encoded, test_size=0.2, random_state=42, stratify=y_encoded
)
```

Support Vector Machine

- Fitting model and calculate accuracy

```
● ● ●  
  
# Create SVM model with Radial kernel (RBF)  
# C = Regularization parameter (Cost)  
svm_model = SVC(kernel='rbf', gamma='scale', C=1.0, random_state=42)  
  
# Train the model  
svm_model.fit(X_train_scaled, y_train)  
  
# Predictions  
y_pred = svm_model.predict(X_test_scaled)  
  
# Evaluate performance  
accuracy = accuracy_score(y_test, y_pred)  
print(f"Accuracy: {accuracy:.4f}")  
print("\nClassification_report:")  
print(classification_report(y_test, y_pred))
```

Support Vector Machine

- Fit model and calculate accuracy

Accuracy: 0.7300

Classification_report:

	precision	recall	f1-score	support
0	0.61	0.28	0.39	60
1	0.75	0.92	0.83	140
accuracy			0.73	200
macro avg	0.68	0.60	0.61	200
weighted avg	0.71	0.73	0.69	200

Support Vector Machine

- Tuning Hyperparameter

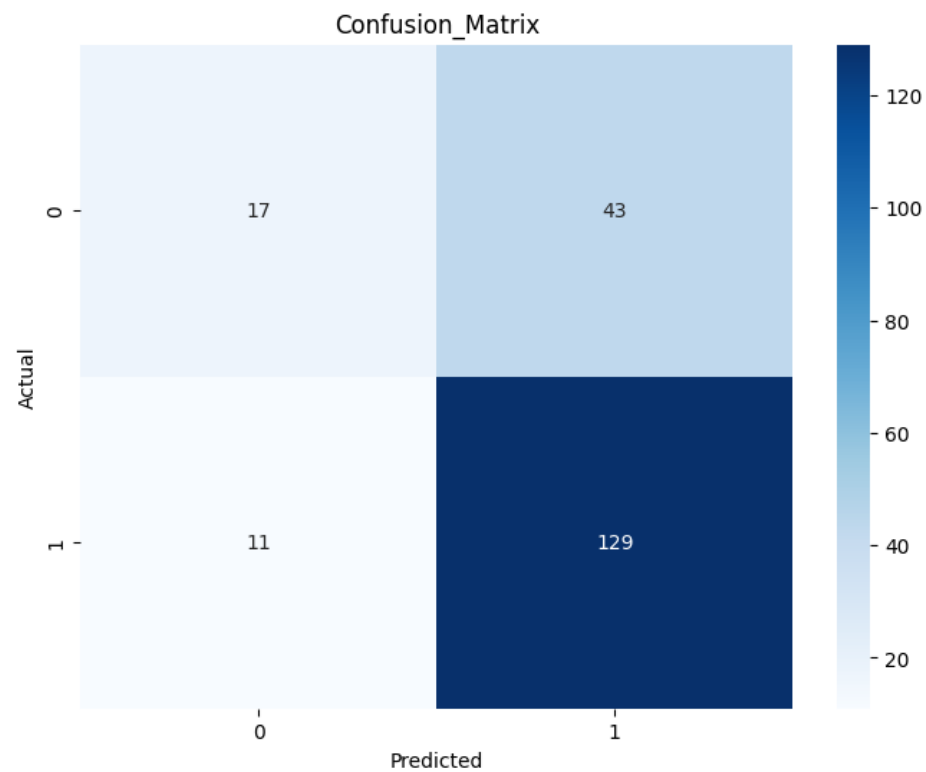
```
● ● ●  
  
# Tuning Hyperparameter  
param_grid = {  
    'C': [0.1, 1, 10, 100],  
    'gamma': ['scale', 'auto', 0.1, 0.01, 0.001]  
}  
  
grid_search = GridSearchCV(  
    SVC(kernel='rbf', random_state=42),  
    param_grid,  
    cv=5,  
    scoring='accuracy',  
    n_jobs=-1  
)
```

- C: 오분류에 대한 penalty 크기
- gamma: RBF 커널과 관련된 값.

Best Hyperparameter:
{'C': 1, 'gamma': 0.001}
best_score: 0.7040

Support Vector Machine

- Confusion matrix



군집분석

- import library



```
from sklearn.datasets import make_blobs
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from scipy.cluster.hierarchy import dendrogram, linkage, fcluster
```

군집분석

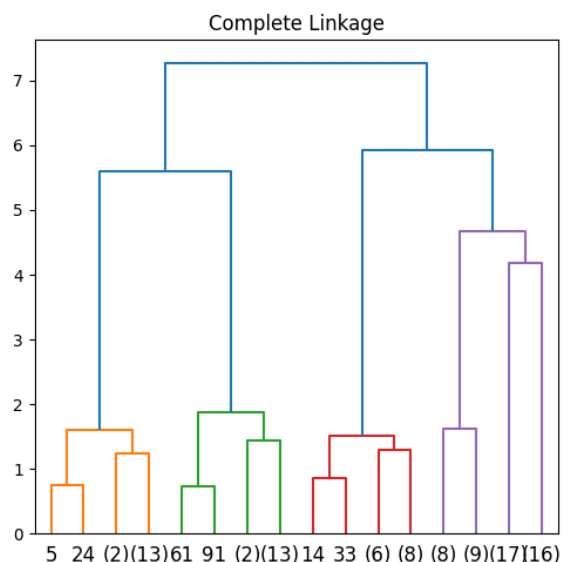
- 모집단 또는 범주에 대한 사전정보가 없는 경우에 주어진 관측값들 사이의 거리 또는 유사성을 이용하여 전체를 몇 개의 집단으로 그룹화하여 각 집단의 성격을 파악, 데이터 전체의 구조에 대한 이해를 돕고자하는 분석법.
- 주어진 자료의 수가 n 일 때, K 개의 군집들은 일반적으로 $C_1, \dots, C_K$ 로 표기하고, $\bigcup_{k=1}^K C_k = \{1, 2, \dots, n\}, C_i \cap C_j = \emptyset, i \neq j$ 을 만족한다.

비유사성 척도

- 관측치들이 서로 얼마나 유사한지 또는 유사하지 않은지를 보통 비유사성을 기준으로 하며 거리를 사용.
- 거리척도의 종류들
 1. 유클리드 거리: $d(x, y) = \sqrt{\sum_{i=1}^p (x_i - y_i)^2}$
 2. Minkowski 거리: $d(x, y) = [\sum_{i=1}^p (x_i - y_i)^m]^{1/m}$
 3. Maximum 거리: $d(x, y) = \max |x_i - y_i|$
 4. Manhattan 거리: $d(x, y) = \sum_{i=1}^p |x_i - y_i|$

계층적 군집분석

- 가까운 자료끼리 차례로 병합해가는 방법으로, 한번 병합된 자료는 다시 분리되지 않는다.
- 군집들이 매 단계 계층적 (나무) 구조를 이루고, 결과를 나무구조인 덴드로그램을 통해 간단히 나타낼 수 있다



계층적 군집분석

●알고리즘

1. 처음 n 개의 자료에 각각 하나의 군집을 할당한다. 즉 초기 군집의 수는 n 이다.
2. 1단계의 n 개의 군집 중 가장 거리가 가까운 두 개의 군집을 병합하여 $n - 1$ 개의 군집으로 군집을 줄인다.
3. 위의 2단계를 반복하여 군집의 수를 줄여나간다.

●군집 간 거리 측정하는 방법 (linkage, 연결법)

1. 최단 연결법 (Single linkage): $d(C_1, C_2) = \min_{i \in C_1, j \in C_2} d(x_i, x_j)$
2. 최장 연결법 (Complete linkage): $d(C_1, C_2) = \max_{i \in C_1, j \in C_2} d(x_i, x_j)$
3. 평균 연결법 (Average linkage): $d(C_1, C_2) = \text{avg}_{i \in C_1, j \in C_2} d(x_i, x_j)$
4. 중점 연결법 (Centroid linkage): $d(C_1, C_2) = d(\bar{x}_1, \bar{x}_2), \bar{x}_i = \text{avg}_{i \in C_i} (x_i)$

계층적 군집분석

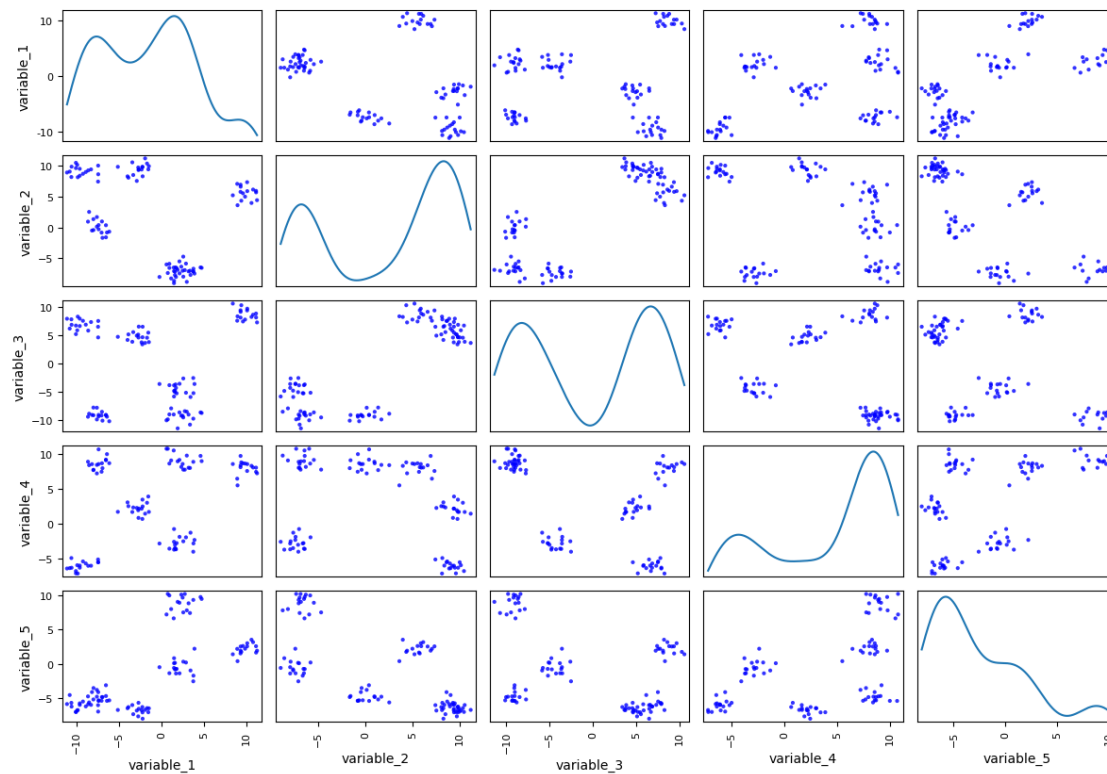
- 10개의 변수를 가진 100개의 데이터, 군집은 총 6가지에서 추출

```
● ● ●  
  
# Setting  
n_samples = 100  
n_features = 10  
  
# Generate Data  
X, y = make_blobs(n_samples=n_samples, n_features=n_features,  
                  centers=6, random_state=42)  
  
# Scaling  
scaler = StandardScaler()  
X_scaled = scaler.fit_transform(X)  
  
# scatter_plot with first 5 variables  
feature_names = [f'variable_{i+1}' for i in range(n_features)]  
df_X = pd.DataFrame(X, columns=feature_names)  
selected_columns = df_X.columns[:5]  
X_selected_df = df_X[selected_columns]  
  
pd.plotting.scatter_matrix(X_selected_df, alpha=0.8, diagonal='kde', figsize=(12, 9),  
                           color='blue')  
plt.suptitle('Multiple Scatter Plot (First 5 Variables)', size=16)  
plt.tight_layout(rect=[0, 0, 1, 0.97])  
plt.show()
```

계층적 군집분석

- 산포도 (scatter plot) 확인

Multiple Scatter Plot (First 5 Variables)



계층적 군집분석

- Dendrogram with average linkage according to various distance methods
- Distance method : Euclidean, Minkowski, Chebyshev, City Block

```
● ● ●  
  
# Dendrogram with average linkage according to Distance method  
fig = plt.figure(figsize=(20, 10))  
  
'''metric = euclidean, minkowski, chebyshev(max), cityblock(Manhattan)'''  
# metric='euclidean'  
ax1 = fig.add_subplot(2, 2, 1)  
dend1 = linkage(X_scaled, metric='euclidean', method='average')  
dendrogram(dend1, ax=ax1, truncate_mode='level', p=3)  
ax1.set_title('Average Linkage with Euclidean Distance ' )  
  
# metric='minkowski'  
...  
  
plt.tight_layout()  
plt.show()
```

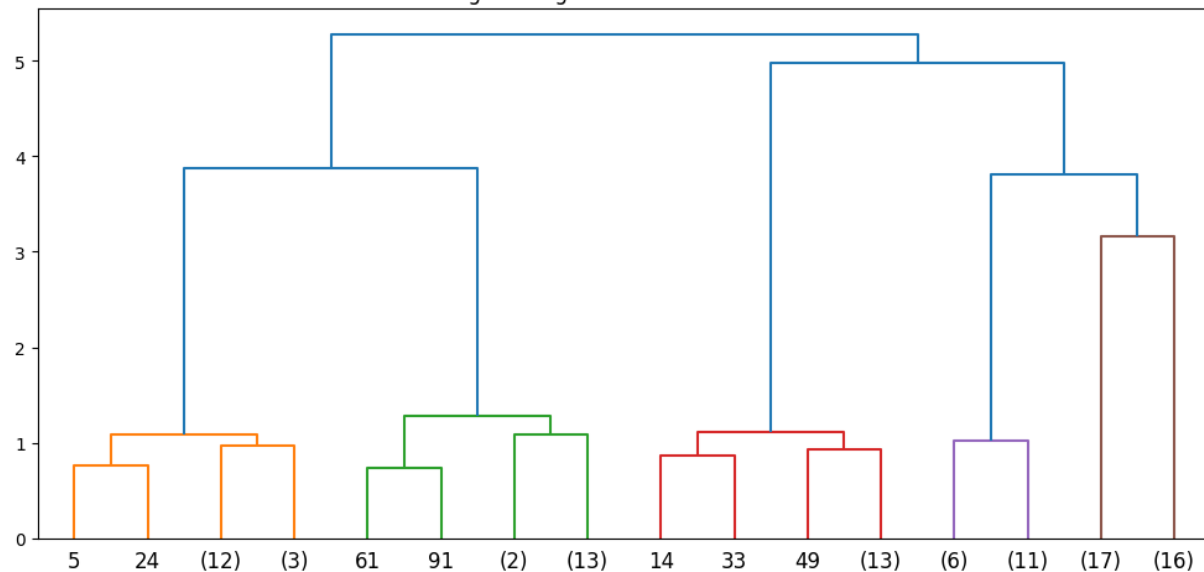
유클리드 거리: $d(x, y) = \sqrt{\sum_{i=1}^p (x_i - y_i)^2}$

Minkowski 거리: $d(x, y) = [\sum_{i=1}^p (x_i - y_i)^m]^{1/m}$

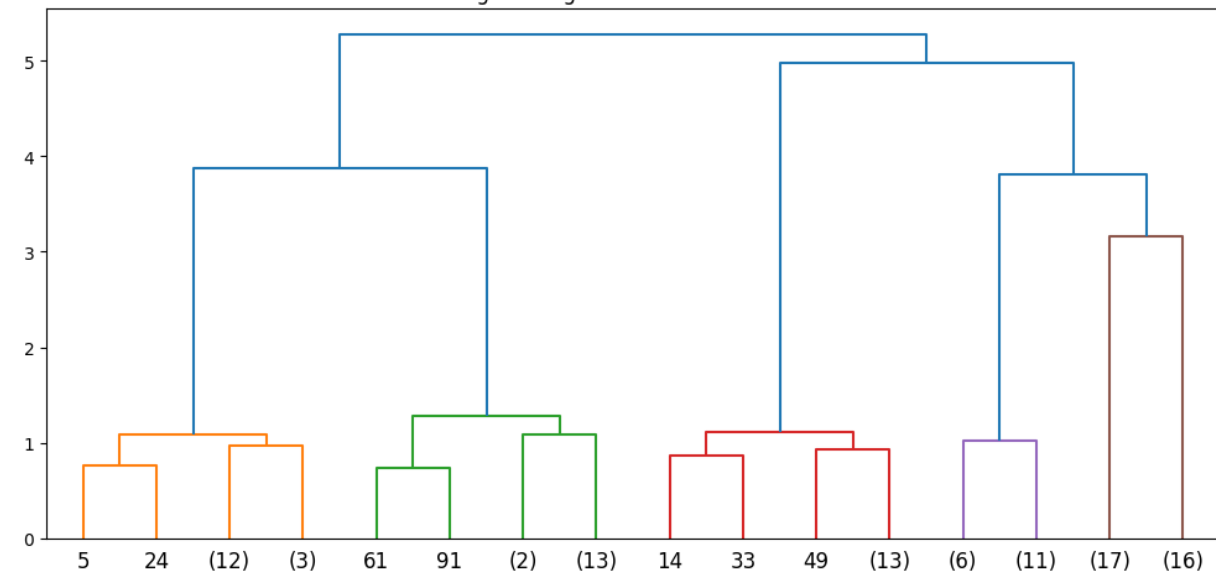
Maximum 거리: $d(x, y) = \max |x_i - y_i|$

Manhattan 거리: $d(x, y) = \sum_{i=1}^p |x_i - y_i|$

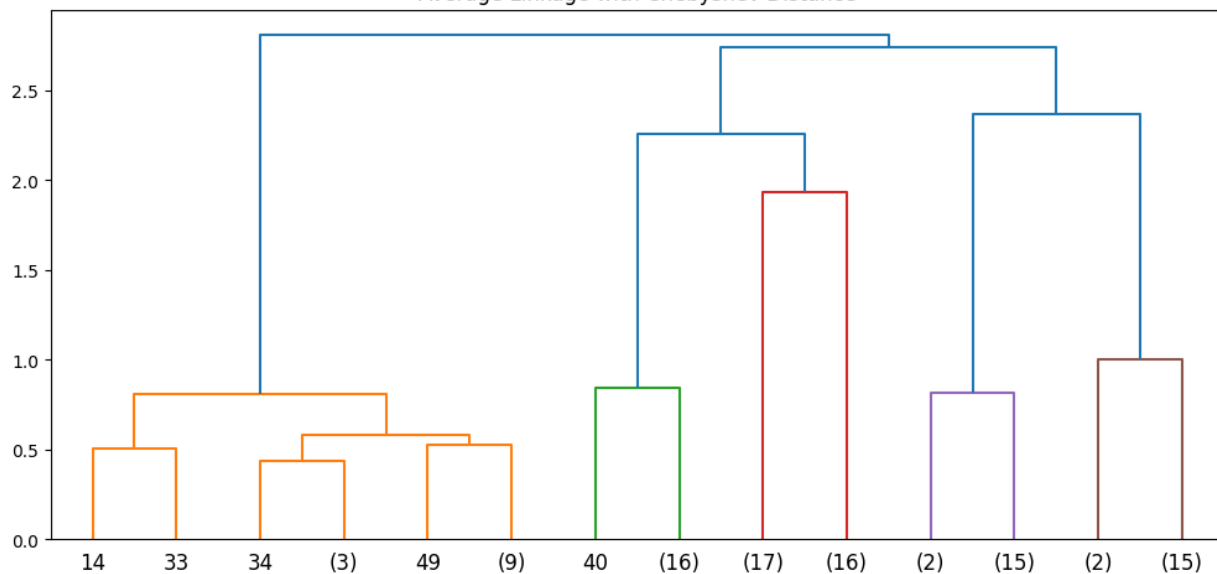
Average Linkage with Euclidean Distance



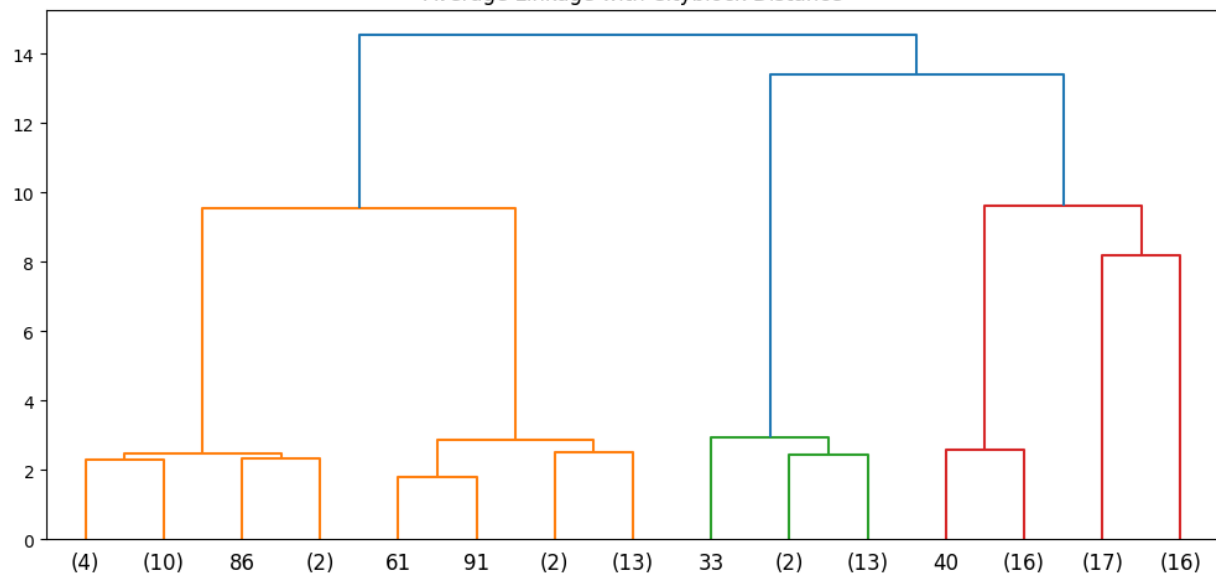
Average Linkage with Minkowski Distance



Average Linkage with Chebyshev Distance

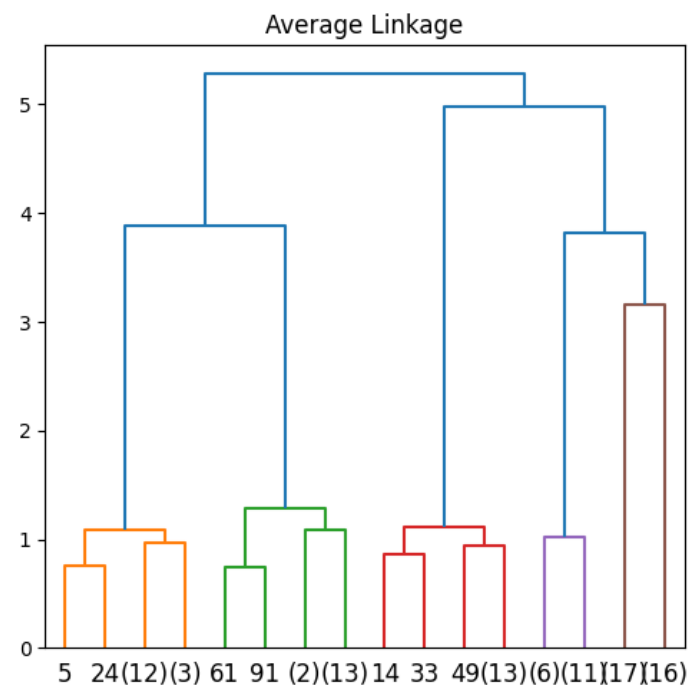
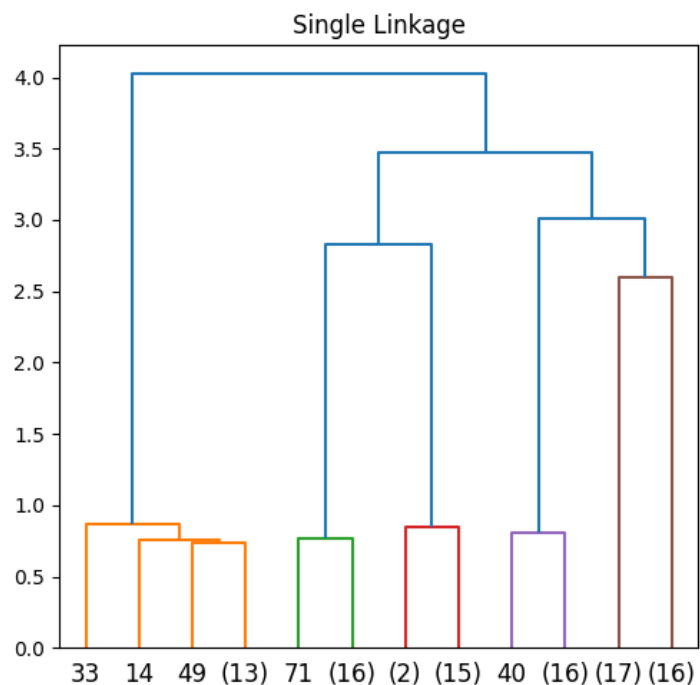
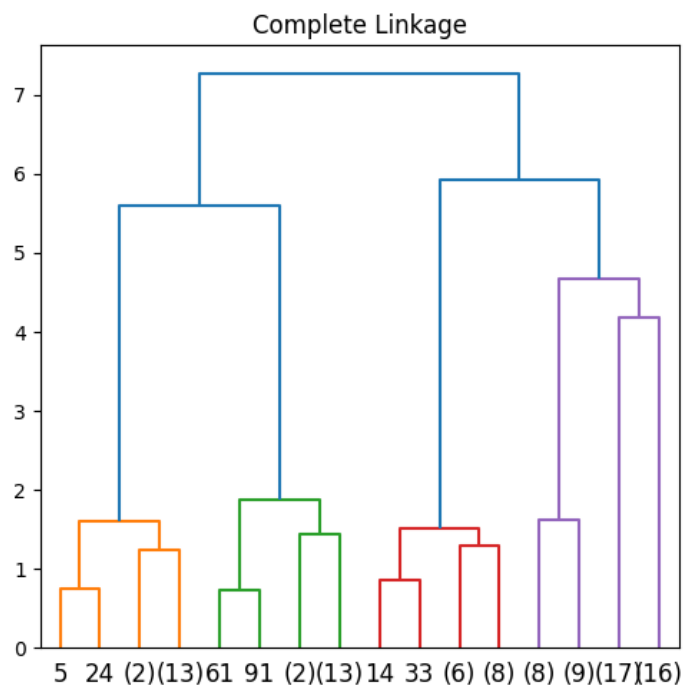


Average Linkage with Cityblock Distance



계층적 군집분석

- Euclidean 거리와 다양한 Linkage 방법으로 구한 dendrogram들



계층적 군집분석

- Data Summary of each linkage method

Summary for Single linkage:

Cluster 1: 16 samples

Key feature means: Feature 1=9.87, Feature 2=5.6, Feature 10=-3.29

Cluster 2: 34 samples

Key feature means: Feature 1=-6.06, Feature 2=9.25, Feature 10=0.08

Cluster 3: 50 samples

Key feature means: Feature 1=-0.81, Feature 2=-4.76, Feature 10=-5.32

Summary for Complete linkage:

Cluster 1: 34 samples

Key feature means: Feature 1=-6.06, Feature 2=9.25, Feature 10=0.08

Cluster 2: 16 samples

Key feature means: Feature 1=9.87, Feature 2=5.6, Feature 10=-3.29

Cluster 3: 50 samples

Key feature means: Feature 1=-0.81, Feature 2=-4.76, Feature 10=-5.32

Summary for Average linkage:

Cluster 1: 34 samples

Key feature means: Feature 1=-6.06, Feature 2=9.25, Feature 10=0.08

Cluster 2: 16 samples

Key feature means: Feature 1=9.87, Feature 2=5.6, Feature 10=-3.29

Cluster 3: 50 samples

Key feature means: Feature 1=-0.81, Feature 2=-4.76, Feature 10=-5.32

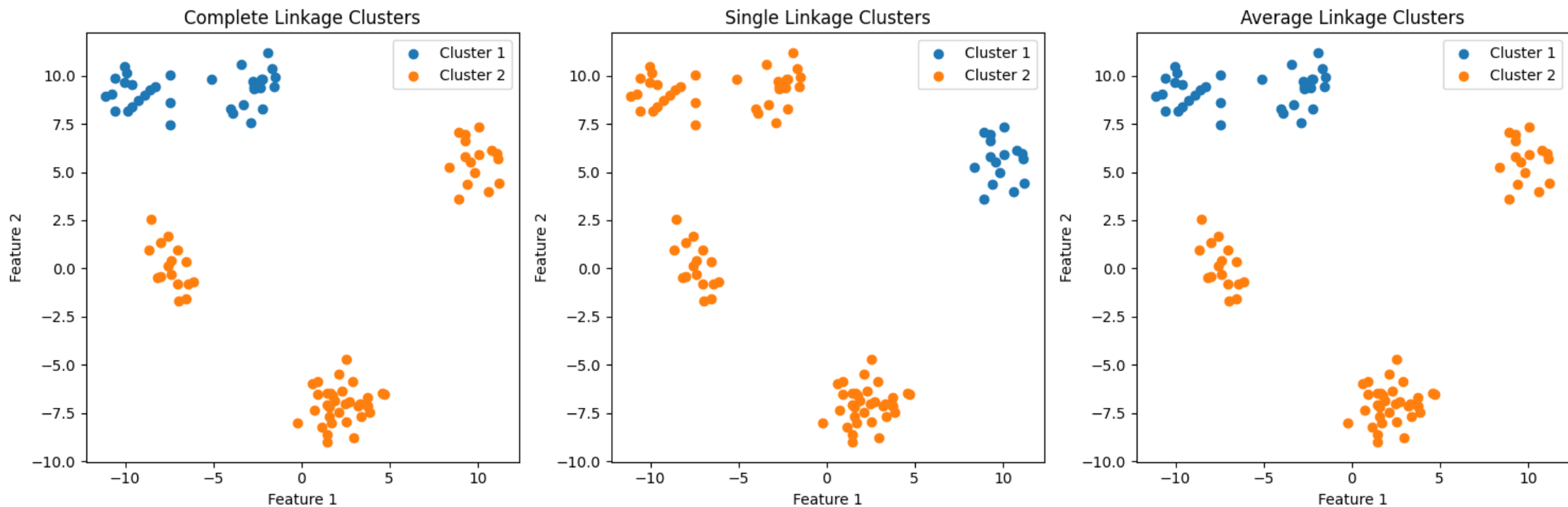
최단 연결법 (Single linkage): $d(C_1, C_2) = \min_{i \in C_1, j \in C_2} d(x_i, x_j)$

최장 연결법 (Complete linkage): $d(C_1, C_2) = \max_{i \in C_1, j \in C_2} d(x_i, x_j)$

평균 연결법 (Average linkage): $d(C_1, C_2) = \text{avg}_{i \in C_1, j \in C_2} d(x_i, x_j)$

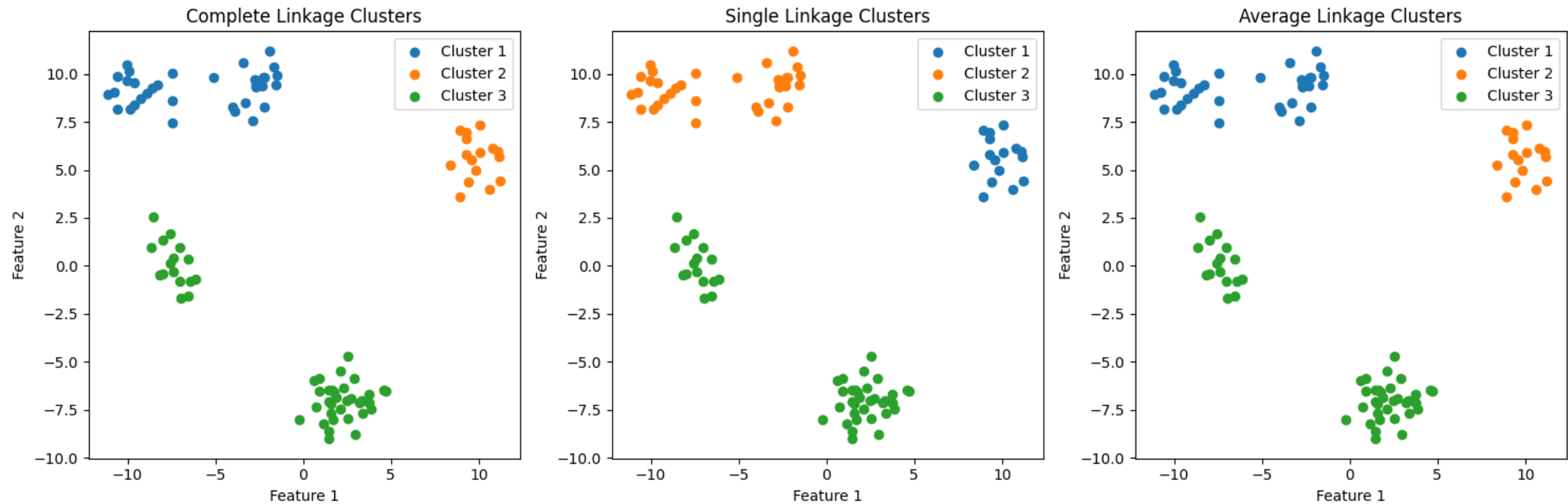
계층적 군집분석

- linkage method에 따른 scatter plot



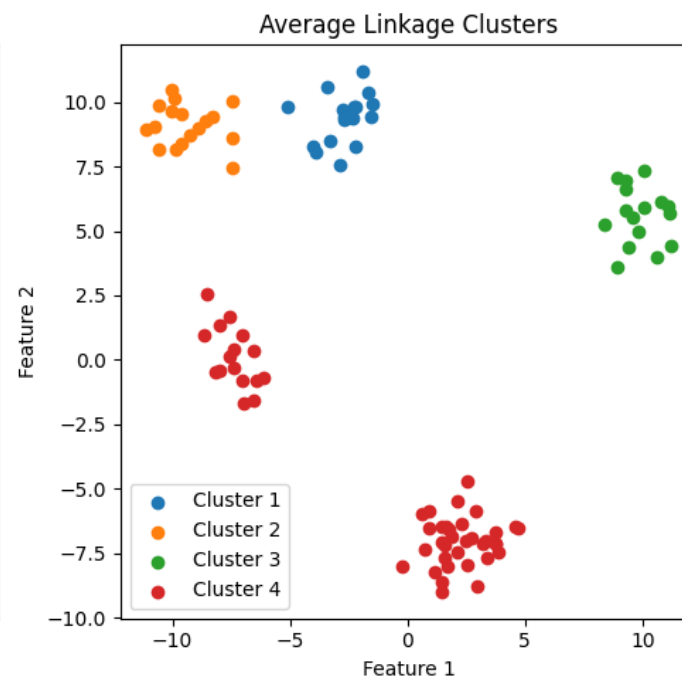
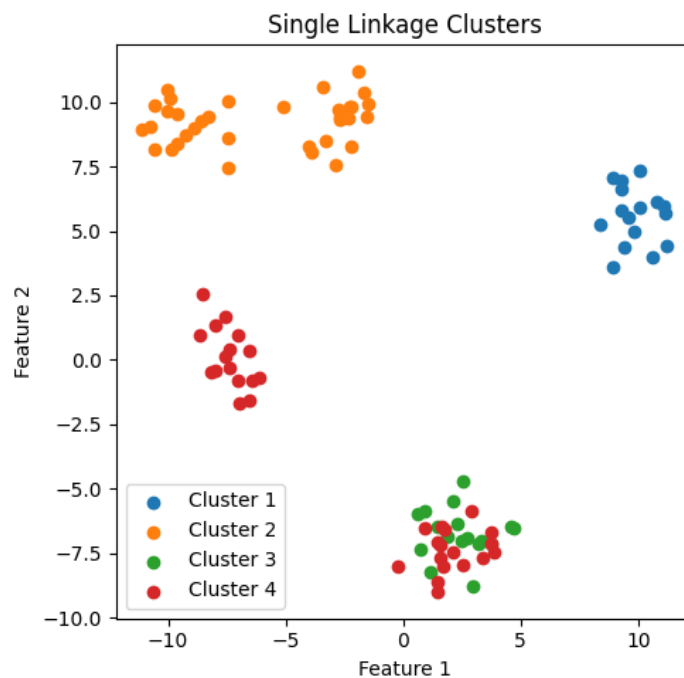
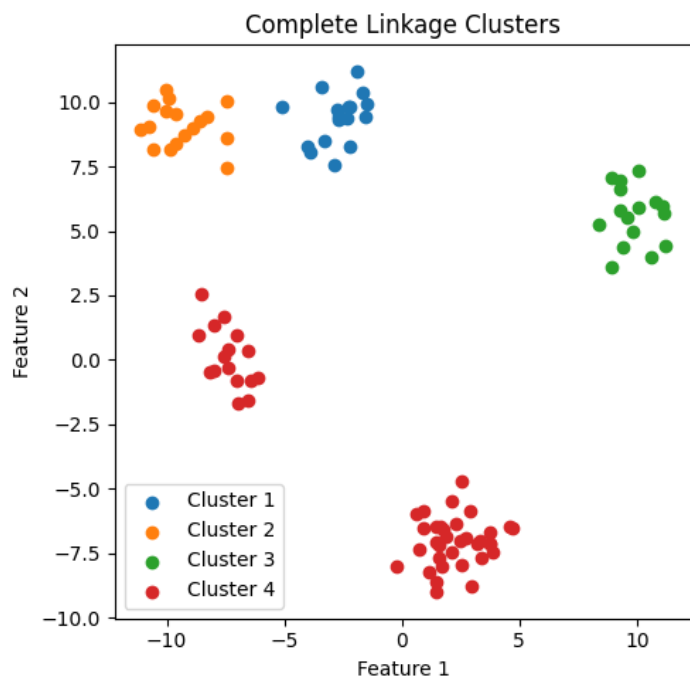
계층적 군집분석

- linkage method에 따른 scatter plot



계층적 군집분석

- linkage method에 따른 scatter plot



K-Means

- 비계층적 군집분석으로, 사전에 결정된 군집 수 K 에 기초하여 전체 데이터를 상대적으로 유사한 K 개의 군집으로 구분.
- 계층적 군집방법에 비해 계산량이 적음.
- 알고리즘
 1. 군집 수 K 을 결정.
 2. 초기 K 개 군집의 중심을 랜덤하게 선택.
 3. 전체 관측 개체를 군집의 중심들과 거리를 계산하여 가까운 거리에 있는 군집으로 할당.
 4. 군집의 중심을 각 군집 내 관측 개체의 평균으로 업데이트.
 5. 3단계와 4단계를 기존의 중심과 새로운 중심의 차이가 없을 때까지 반복.

K-Means

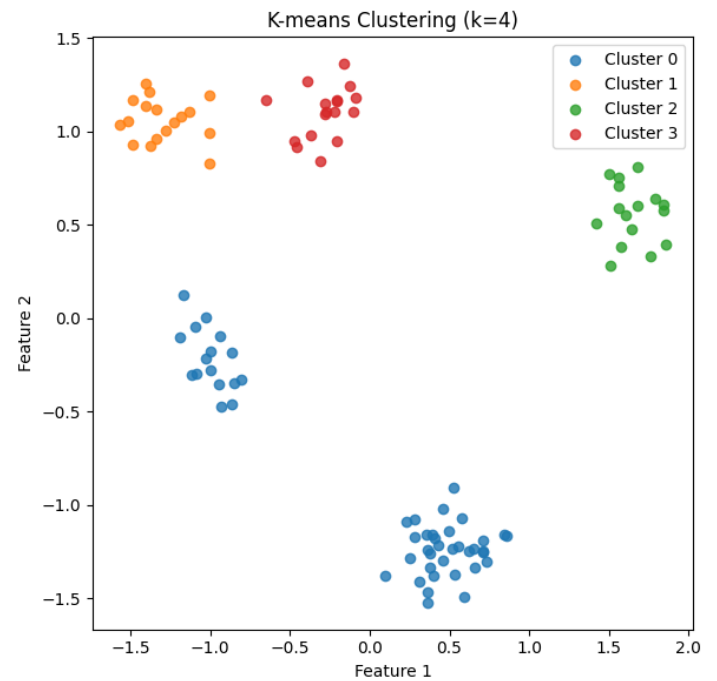
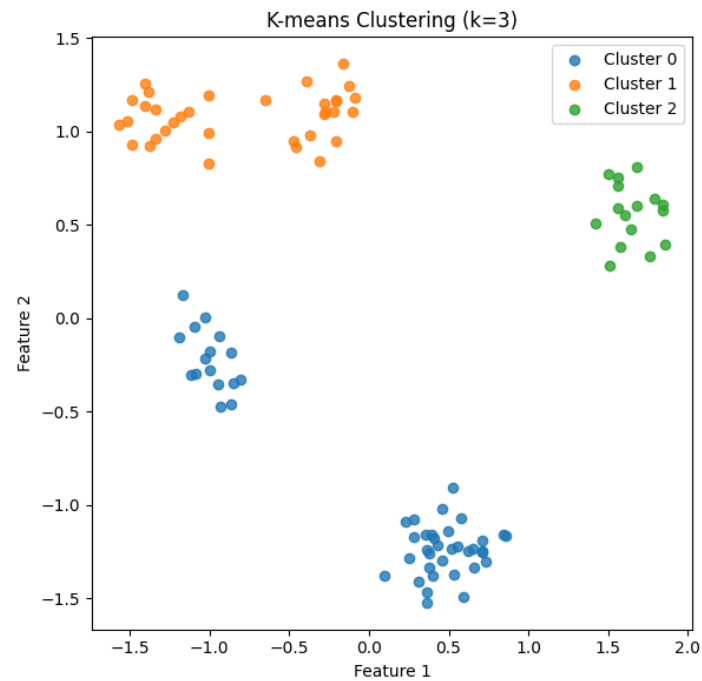
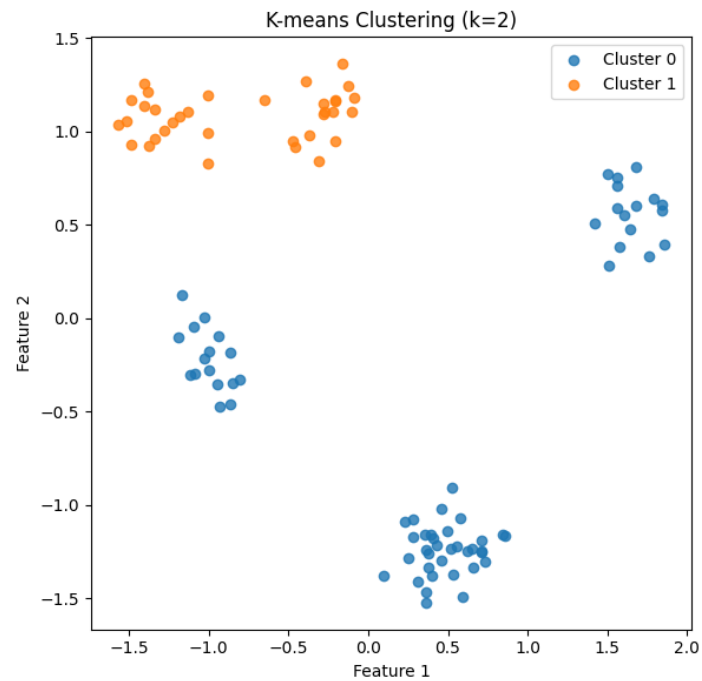
- sklearn.cluster 안의 KMeans 함수 사용



```
kmeans = KMeans(n_clusters=k, random_state=42)
```

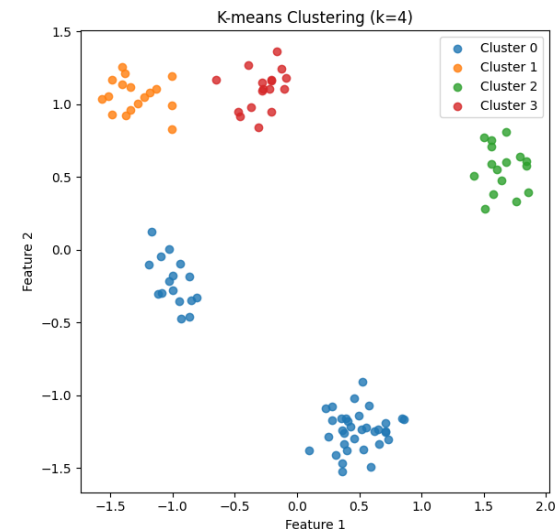
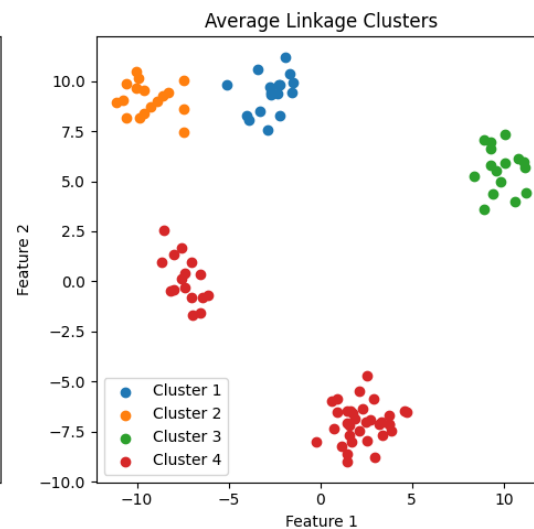
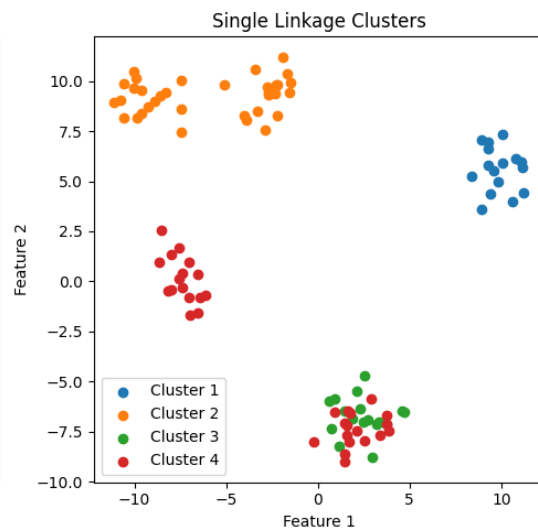
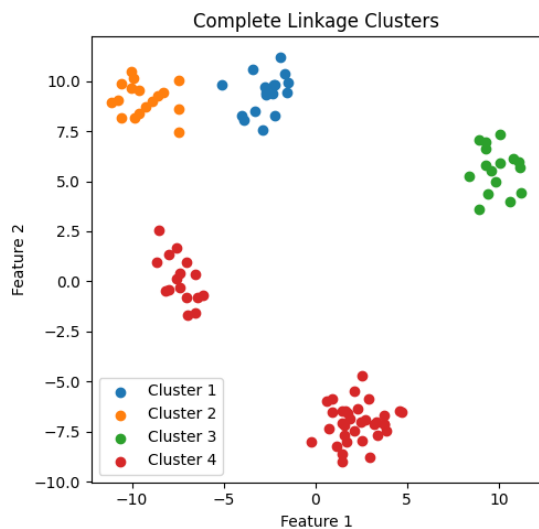
```
kmeans.fit(X_scaled)
```

K-Means



K-Means

● 계층적 군집분석 vs. KMeans



K-Means

- 군집 수 K 의 결정

```
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt

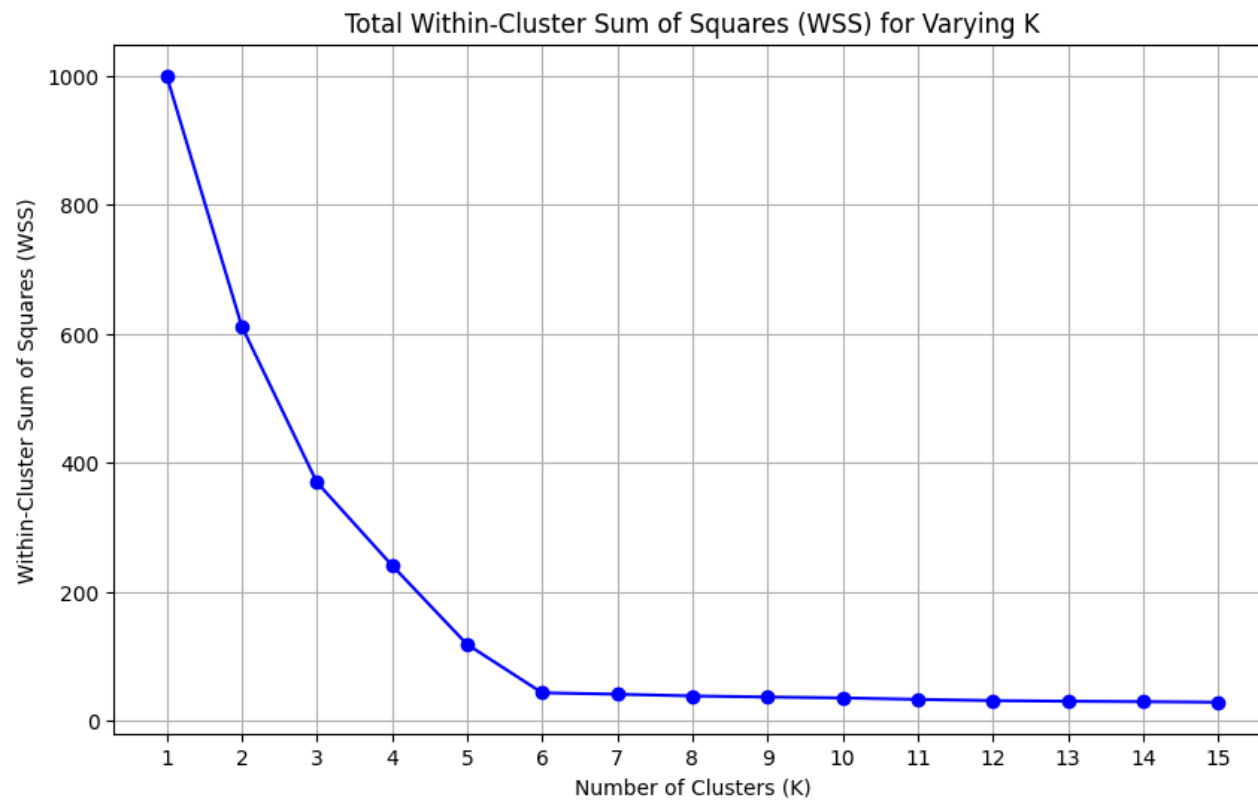
# WSS
wss = []

# from K=1 to K=15
for k in range(1, 16):
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    kmeans.fit(X_scaled)
    wss.append(kmeans.inertia_)

# Plot
plt.figure(figsize=(10, 6)) ...
```

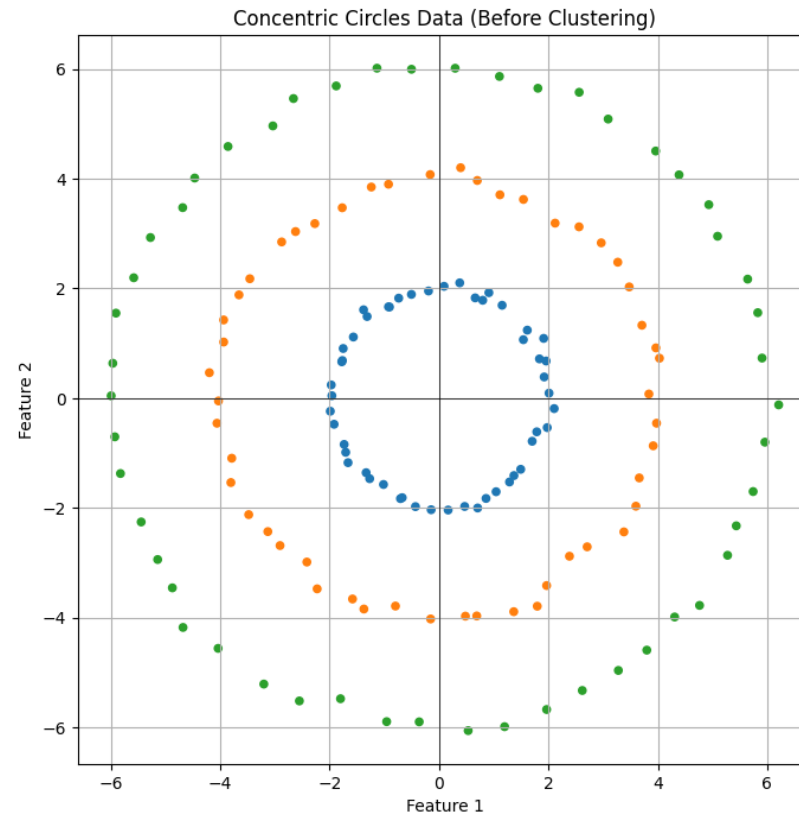
K-Means

- 군집 수 K 의 결정



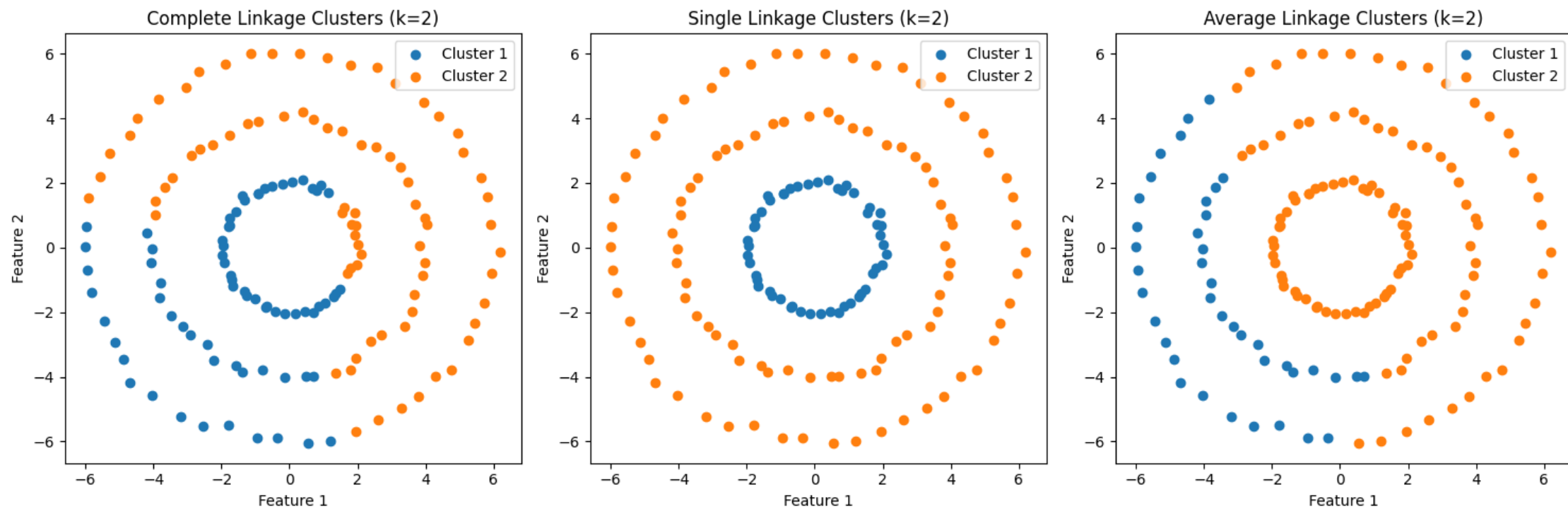
K-Means

- Clustering for Bad Data



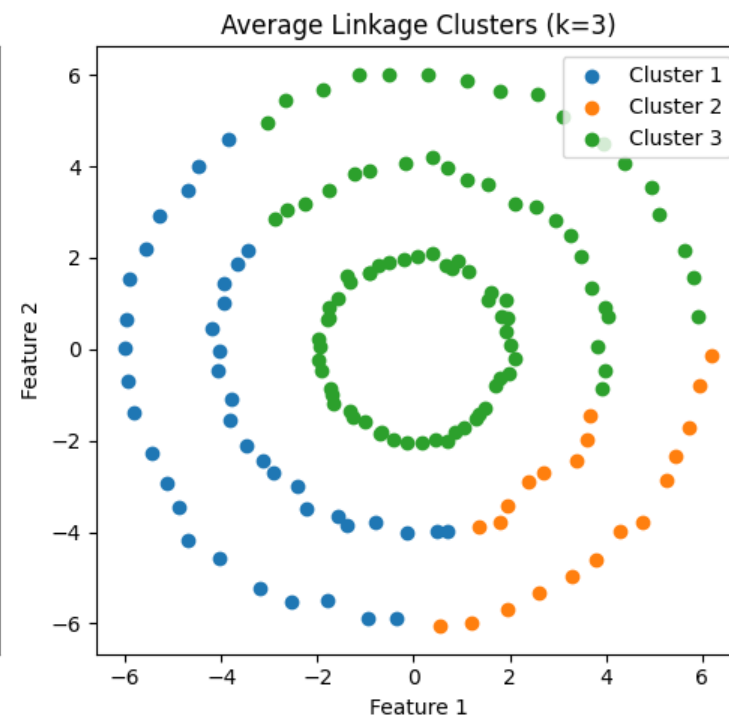
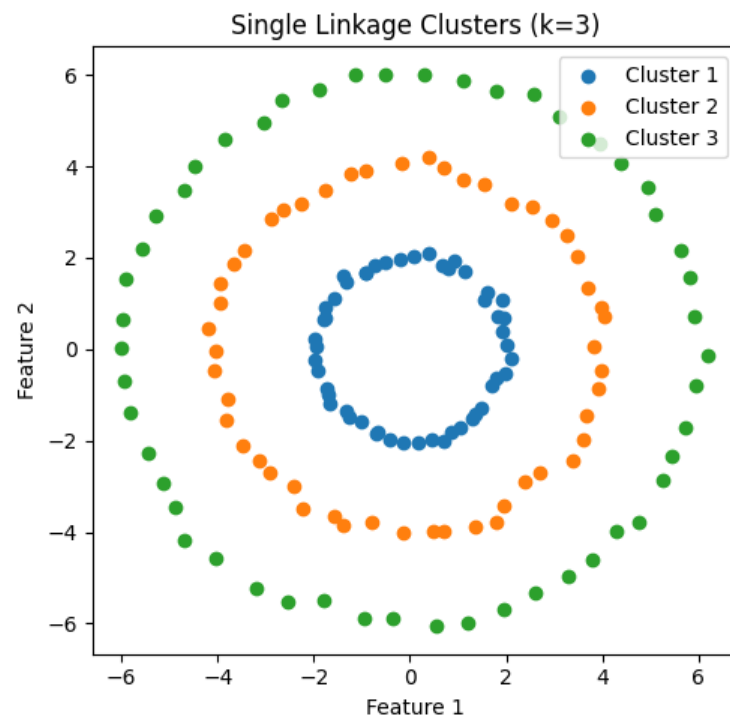
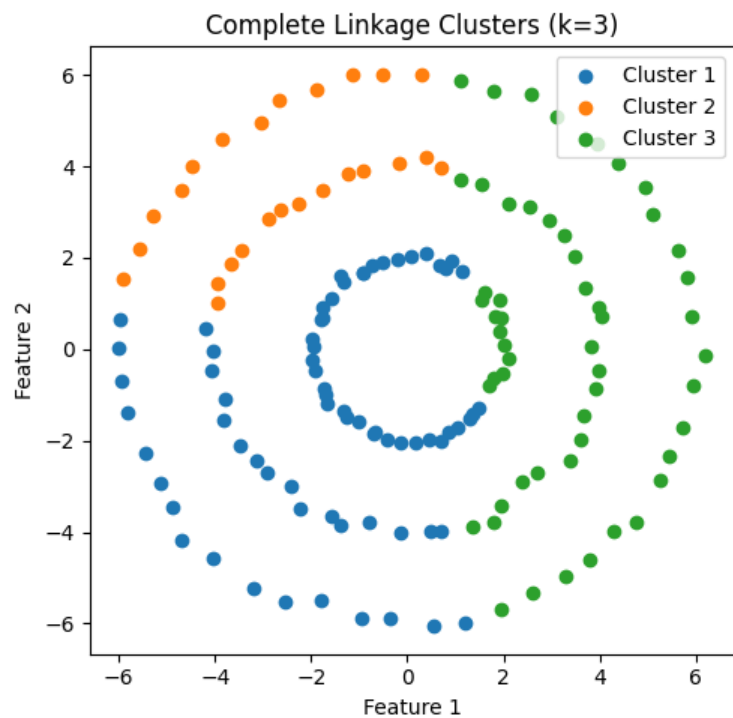
K-Means

- Clustering for Bad Data



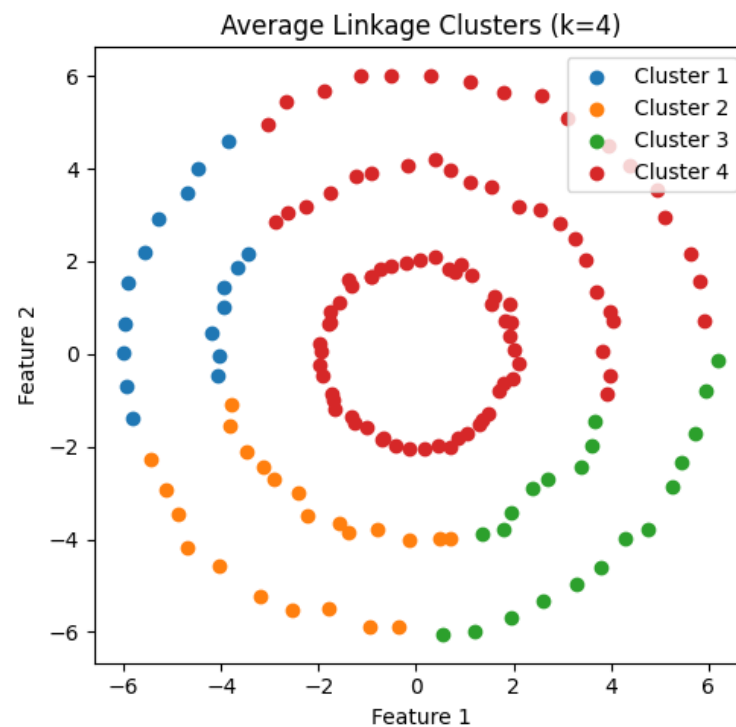
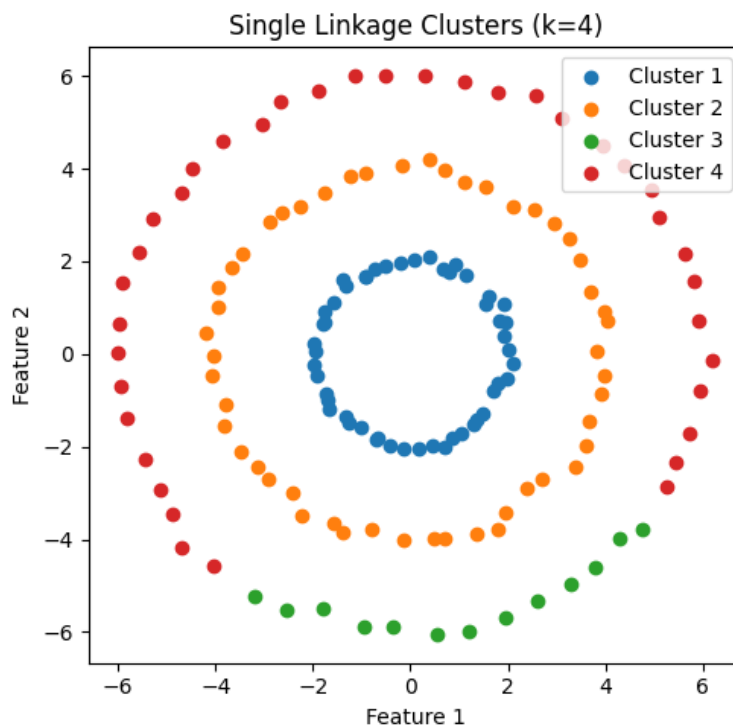
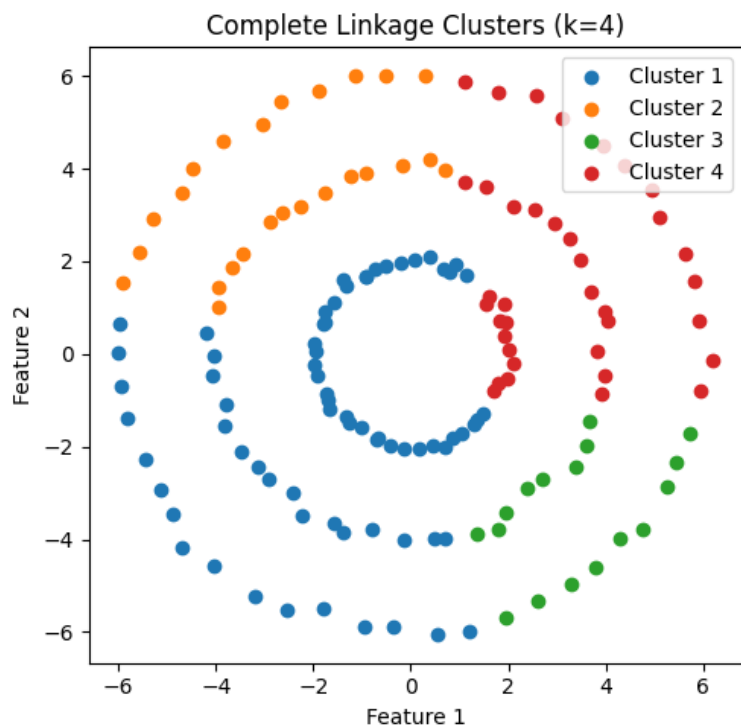
K-Means

- Clustering for Bad Data



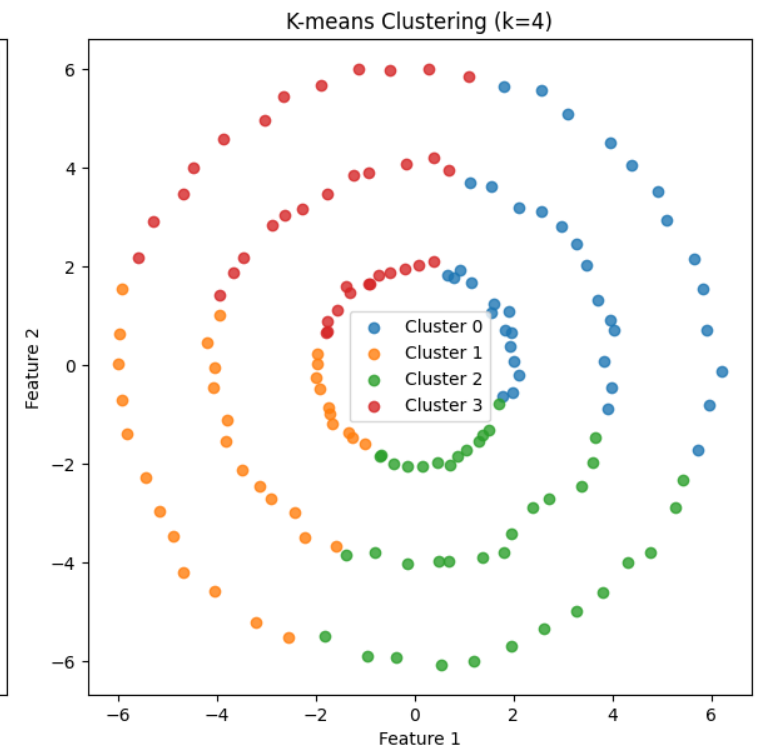
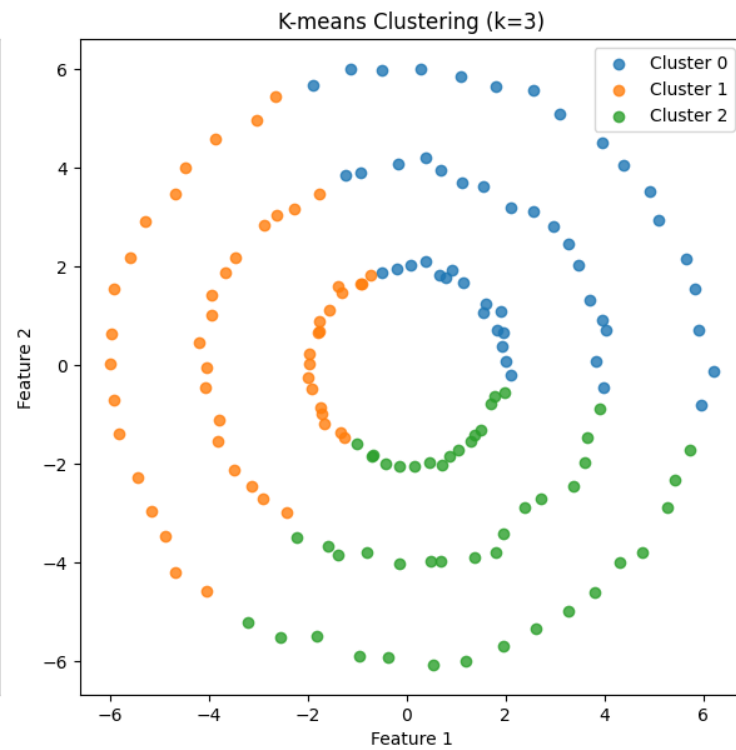
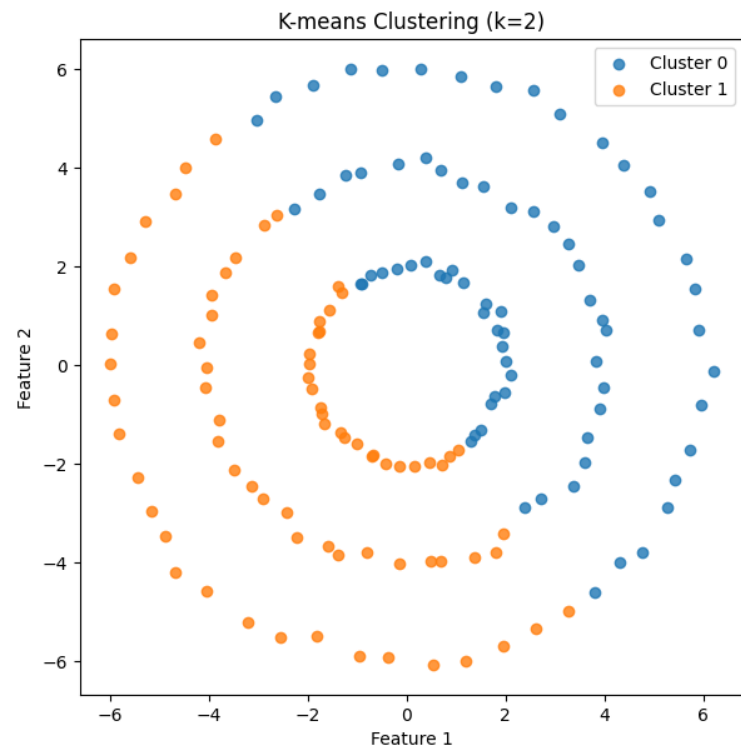
K-Means

- Clustering for Bad Data



K-Means

- Clustering for Bad Data



직접 해보기 – iris dataset

1. iris 데이터를 불러온다.
2. 범주형 변수를 제외한 데이터 생성한다.
3. StandardScale함수를 통해서 Data Standardize한다.
4. 유클리디안 거리와 군집 연결법을 ward로 하여 linkage 객체를 생성한다.
5. truncate_mode='level', p=3 Parameter를 사용하여 dendrogram을 작성한다.



```
# 1
iris_data = sns.load_dataset("iris")

# 2
iris_numeric = iris_data.select_dtypes(include=["float64", "int64"])

# 3
scaler = StandardScaler()
scaled_data = scaler.fit_transform(DATA)
```

직접 해보기 - iris dataset

6. Linkage 객체를 사용하여 군집 3개의 클러스터를 생성한다.
7. `sns.pairplot`을 사용하여 각 변수들의 scatter plot을 그리고 군집별로 색을 달리한다.

차원축소 - PCA

- 가장 큰 분산을 가지는 축을 기준으로 projection 하는 방법.
- 정보 손실을 최소화하며 차원을 축소할 수 있다.
- PCA의 목적함수

$$\max_{\|w\|=1} w^T \Sigma w, \quad (\Sigma w = \lambda w)$$

- w : 데이터 투영 방향을 나타내는 단위 벡터
- Σ : 입력 데이터의 공분산 행렬
- λ : Σ 의 고유값, 해당 방향의 분산 크기를 의미

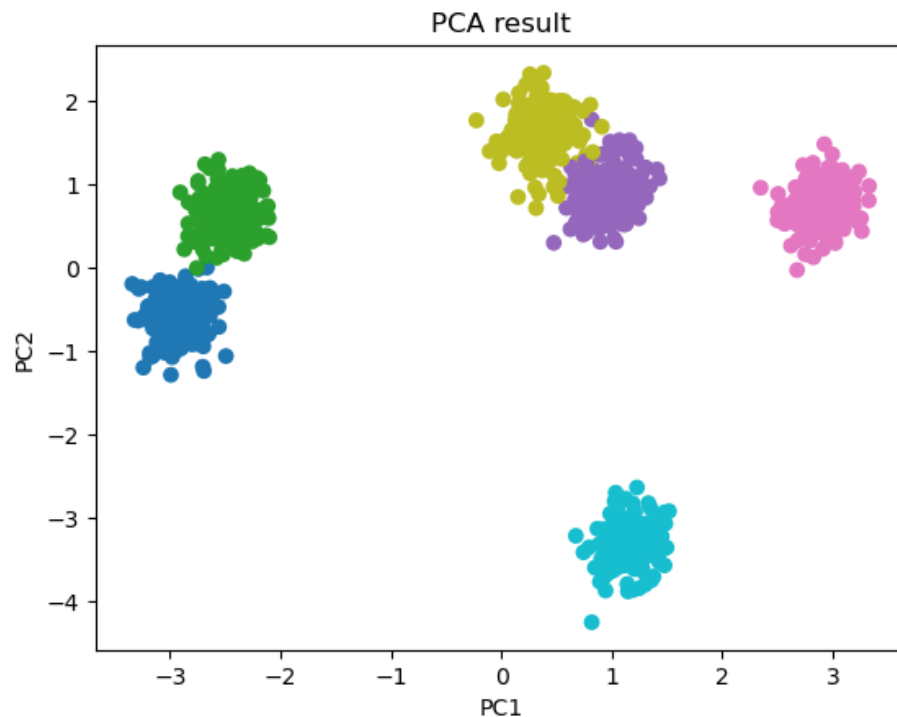
차원축소 - PCA

- 가장 큰 분산을 가지는 축을 기준으로 projection 하는 방법.
- 정보 손실을 최소화하며 차원을 축소할 수 있다.

```
# 데이터 준비
X, y = make_blobs(n_samples=1000, n_features=10, centers=6, random_state=42)
X_scaled = StandardScaler().fit_transform(X)

# PCA 적용
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

# 시각화
plt.scatter(X_pca[:,0], X_pca[:,1], c=y, cmap='tab10')
plt.xlabel('PC1'); plt.ylabel('PC2'); plt.title('PCA result')
plt.show()
```



차원축소 - MDS

- 데이터 사이의 거리를 보존하도록 차원을 축소하는 방법.
- 차원 축소 후, 데이터 간 상대적 위치 관계가 유지된다.
- MDS의 목적함수

$$\min_Y \sum_{i < j} (d_{ij} - \|y_i - y_j\|)^2$$

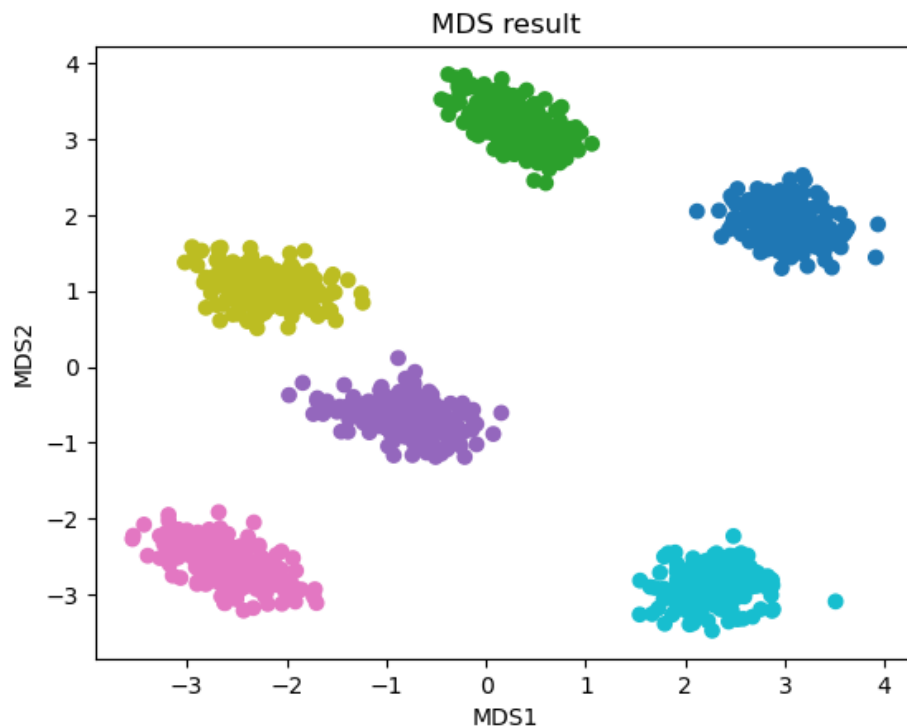
- $Y = [y_1, \dots, y_n]^\top$: 저차원 좌표 행렬
- d_{ij} : 원본 고차원에서 점 i 와 j 사이의 유클리드 거리
- y_i, y_j : 저차원 공간에서 점 i 와 j 의 좌표 벡터

차원축소 - MDS

- 데이터 사이의 거리를 보존하도록 차원을 축소하는 방법.
- 차원 축소 후, 데이터 간 상대적 위치 관계가 유지된다.

```
# MDS 적용
mds = MDS(n_components=2, random_state=42)
X_mds = mds.fit_transform(X_scaled)

# 시각화
plt.scatter(X_mds[:,0], X_mds[:,1], c=y, cmap='tab10')
plt.xlabel('MDS1'); plt.ylabel('MDS2'); plt.title('MDS result')
plt.show()
```



차원축소 - T-SNE

- 각 데이터끼리의 유사도 분포를 유지하도록 차원을 축소하는 방법.
- T-SNE의 목적함수

$$C = \sum_{i \neq j} p_{ij} \log \frac{p_{ij}}{q_{ij}}$$

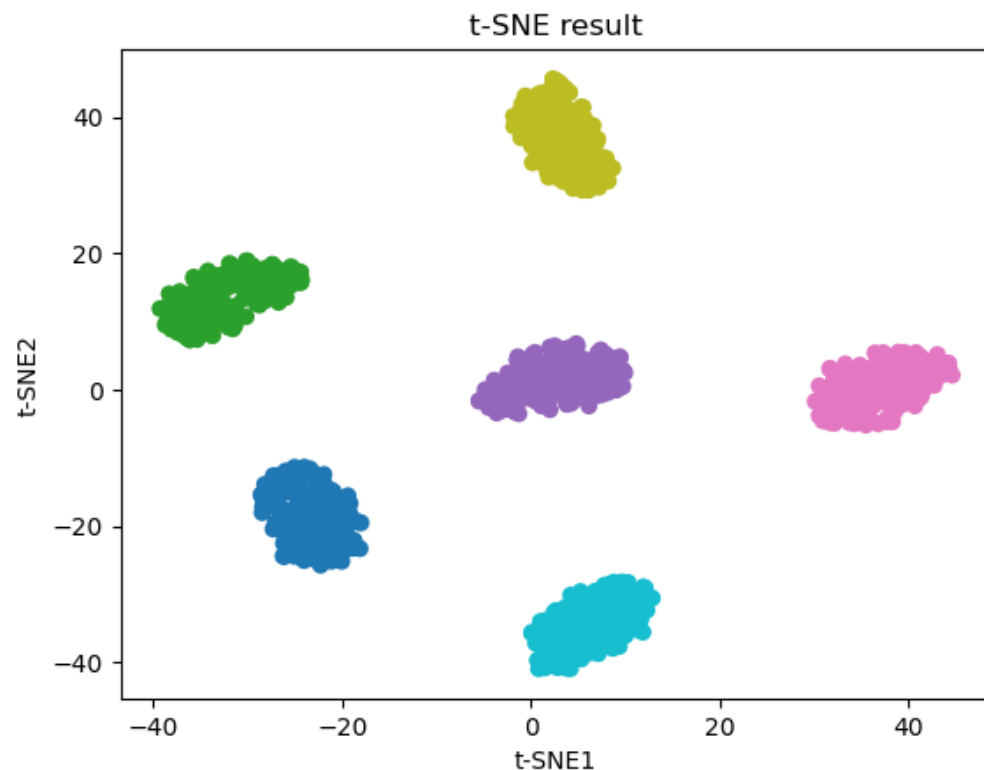
- C : KL 발산 비용 함수
- p_{ij} : 고차원에서의 이웃 유사도 확률
- q_{ij} : 저차원에서의 이웃 유사도 확률

차원축소 - T-SNE

- 각 데이터끼리의 유사도 분포를 유지하도록 차원을 축소하는 방법.

```
# t-SNE 적용
tsne = TSNE(n_components=2, random_state=42)
X_tsne = tsne.fit_transform(X_scaled)

# 시각화
plt.scatter(X_tsne[:,0], X_tsne[:,1], c=y, cmap='tab10')
plt.xlabel('t-SNE1'); plt.ylabel('t-SNE2'); plt.title('t-SNE result')
plt.show()
```



서로 다른 모형 비교하기

- 차원축소 후, 로지스틱 회귀 모형 적합
- 원본 데이터로 로지스틱 회귀 모형 적합
- 원본 데이터로 SVM 모형 적합

```
# 2. PCA → 2차원 축소 + Logistic
pca = PCA(n_components=2, random_state=42)
X_pca = pca.fit_transform(X_scaled)
X_train_p, X_test_p, y_train_p, y_test_p = train_test_split(
    X_pca, y, test_size=0.3, random_state=42, stratify=y
)
log_pca = LogisticRegression(max_iter=1000, random_state=42).fit(X_train_p, y_train_p)
y_pred_pca = log_pca.predict(X_test_p)
```

```
# 3. MDS → 2차원 축소 + Logistic
mds = MDS(n_components=2, random_state=42)
X_mds = mds.fit_transform(X_scaled)
X_train_m, X_test_m, y_train_m, y_test_m = train_test_split(
    X_mds, y, test_size=0.3, random_state=42, stratify=y
)
log_mds = LogisticRegression(max_iter=1000, random_state=42).fit(X_train_m, y_train_m)
y_pred_mds = log_mds.predict(X_test_m)
```

```
# 5. 원본 데이터로 Logistic & SVM
X_train_s, X_test_s, y_train_s, y_test_s = train_test_split(
    X_scaled, y, test_size=0.3, random_state=42, stratify=y
)
log = LogisticRegression(max_iter=1000, random_state=42).fit(X_train_s, y_train_s)
svm = SVC(kernel='rbf', random_state=42).fit(X_train_s, y_train_s)
y_pred_log = log.predict(X_test_s)
y_pred_svm = svm.predict(X_test_s)
```

	accuracy	recall	precision
Logistic (PCA)	0.730000	0.928571	0.747126
Logistic (MDS)	0.700000	1.000000	0.700000
Logistic (t-SNE)	0.710000	0.966667	0.717314
Logistic (원본 데이터)	0.736667	0.842857	0.793722
SVM (원본 데이터)	0.740000	0.914286	0.761905

서로 다른 모형 비교하기

- 3가지 방식으로 차원축소 후, 로지스틱 회귀로 분류하고 시각화한다.

